```
# Allow apt to work with https-based sources
RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
  apt-transport-https

# Ensure we install an up-to-date version of Node
# See https://github.com/yarnpkg/yarn/issues/2888
RUN curl -sL https://deb.nodesource.com/setup_8.x | bash -

# Ensure latest packages for Yarn
RUN curl -sS https://dl.yarnpkg.com/debian/pubkey.gpg | apt-key add -
RUN echo "deb https://dl.yarnpkg.com/debian/ stable main" | \
  tee /etc/apt/sources.list.d/yarn.list

# Install packages
RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
  nodejs \
  yarn

COPY Gemfile* /usr/src/app/
WORKDIR /usr/src/app
RUN bundle install

COPY . /usr/src/app/

CMD ["bin/rails", "s", "-b", "0.0.0.0"]
```

To install an up-to-date version of Yarn, we have to add Yarn's Debian package repository to the list of sources (lines 13–15). However, since Yarn's package repo uses HTTPS, we have to install the apt-transport-https package (lines 5–6) to allow this to work.

Unfortunately, there's a dependency issue between Yarn and the (old) version of Node.js installed by default. We solve this on line 10 by adding Node's package repository to the list of sources; this ensures we install a more up-to-date version of Node.

Finally, on line 20, we add yarn to the list of packages we install. With yarn and an up-to-date version of Node installed, we're now ready to configure our app to use Webpacker.

Had we known we were going to be using Webpacker when first creating our app, we could have included support using the --webpack option—for example:

```
$ rails new myapp --webpack=react <other options>
```

The --webpack=react option would have generated our app with support for React out of the box. However, having already generated our app, adding Webpacker support requires a couple of manual steps.